“Linux Signal-Based Process Communication and Control System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

R. RADHAKRISHNA	**2520090063**
R. JAGADEESH	**2520090030**
P. VEDADRI NAIDU	**2520090060**

Under the Esteemed Guidance of

Raghupathi M

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **"Linux Signal-Based Process Communication and Control System"** is a record of Bonafide work of **R. RADHAKRISHNA - 2520090063, R. JAGADEESH - 2520090030, P. VEDADRI NAIDU - 2520090060** submitted in
partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

R. RADHAKRISHNA	**2520090063**
R. JAGADEESH	**2520090030**
P. VEDADRI NAIDU	**2520090060**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled**"Linux Signal-Based Process Communication and Control System"** is a bonafide work done and submitted by **R. RADHAKRISHNA - 2520090063, R. JAGADEESH - 2520090030, P. VEDADRI NAIDU - 2520090060,**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our guide Raghupathi M, Department of Computer Science and Engineering, which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

R. RADHAKRISHNA 2520090063

R. JAGADEESH 2520090030

P. VEDADRI NAIDU 2520090060

Abstract

Modern distributed systems require efficient inter-process communication mechanisms to coordinate activities between multiple processes. This project aims to develop a Linux Signal-Based Process Communication and Control System that leverages POSIX signals for real-time process synchronization and control. The system addresses the challenge of managing multiple processes with varying computational loads and priorities while maintaining low-latency communication. Traditional IPC mechanisms like pipes and message queues introduce overhead; signals offer a lightweight alternative for event-driven communication.

The proposed solution implements a signal handler framework that processes various system signals (SIGUSR1, SIGUSR2, SIGALRM) to coordinate process states, manage resource allocation, and handle process termination gracefully. The system demonstrates advanced OS concepts including signal masking, signal delivery mechanisms, and process state management. We will implement a master-slave architecture where the master process monitors system resources and sends control signals to slave processes. Key deliverables include a modular C implementation with comprehensive error handling, a signal routing mechanism, and a monitoring dashboard showing real-time process communication.

The expected outcome is a robust, scalable system capable of managing up to 100 concurrent processes with sub-millisecond signal delivery latency. This project provides practical insights into low-level process communication, signal handling semantics, and building efficient systems-level applications on Linux/POSIX platforms.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		6–12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	**Implementation**		**30–45**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**62–67**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**79–83**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**84–86**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER - 1 INTRODUCTION

1.1 Background of the Project

Signals are software interrupts that provide a way to handle asynchronous events. They are one of the oldest methods of inter-process communication (IPC) in Unix/Linux. Modern distributed systems require efficient IPC mechanisms to coordinate activities between multiple processes, and POSIX signals offer a lightweight, event-driven way to do so.

Key concepts:

- **Signal:** a notification sent to a process to indicate that an event has occurred.
- **Signal Handler:** a function that is called when a specific signal is received.
- **Signal Mask:** blocks or unblocks specific signals from being delivered to a process.

Common signals used:

- **SIGUSR1, SIGUSR2:** user-defined signals for custom communication.
- **SIGALRM:** timer signal used for periodic monitoring and timeouts.
- **SIGCHLD:** sent to the parent when a child process terminates.
- **SIGTERM, SIGKILL:** process termination signals.

1.2 Problem Statement

In modern operating systems, efficient IPC is crucial for building robust and scalable applications. While traditional IPC mechanisms like pipes, message queues, and sockets are reliable, they often introduce significant overhead for simple event notifications and process synchronization. Many systems require low-latency communication for control signals, status updates, and process coordination. POSIX signals, though powerful and lightweight, are often underutilized or misunderstood in systems programming.

Developers frequently face challenges in:

1. implementing efficient signal handlers that work reliably with multiple processes,
2. managing signal masks and delivery mechanisms correctly,
3. coordinating multiple processes with varying priorities, and
4. monitoring process states in real time.

This project addresses these challenges by developing a comprehensive signal-based IPC framework that demonstrates best practices in signal handling, process synchronization, and resource management.

1.3 Scope of the Project

The project covers the design and implementation of a Linux Signal-Based Process Communication and Control System in standard C using POSIX APIs. Its scope includes:

- A signal handler framework for SIGUSR1 (status request), SIGUSR2 (shutdown), SIGALRM (periodic monitoring / timeout) and SIGTERM (graceful termination).
- A master-slave process architecture in which the master spawns and controls up to 100 concurrent slave processes.
- A monitoring dashboard that reads /proc/PID/stat and reports CPU usage, memory consumption and signal delivery statistics in real time.
- Handling of edge cases such as orphaned processes, signal coalescing and graceful shutdown through coordinated SIGTERM delivery.
- Testing under various load conditions, signal delivery patterns and process counts.

1.4 Objectives of the Project

1. Implement a robust signal handling framework that correctly manages POSIX signals (SIGUSR1, SIGUSR2, SIGALRM, SIGTERM) with proper signal masking and delivery semantics.
2. Design and develop a master-slave process architecture where the master coordinates multiple slave processes through signal-based communication protocols.
3. Create a process monitoring system that tracks CPU usage, memory consumption, and signal delivery statistics in real time using the /proc filesystem and signal counters.
4. Demonstrate advanced OS concepts including signal atomicity, critical sections, async-signal-safe functions, and race condition prevention in signal handlers.

1.5 Importance of the Application

- Signal-based IPC is a simple and lightweight mechanism with far less overhead than pipes and message queues for simple event notifications.
- It is widely supported in all Unix-like systems and efficient for simple communication patterns.
- The system shows how signals can be used to build responsive, low-latency applications that scale to many concurrent processes while maintaining system stability.
- It gives practical insight into low-level process communication, signal handling semantics and systems-level programming on Linux/POSIX platforms.

1.6 Target Users / Audience

- Students and instructors studying Operating Systems and Systems Programming.
- Systems programmers and developers who need lightweight, event-driven process

coordination on Linux/POSIX platforms.

- Anyone who wants a reference implementation of correct signal handling, signal masking and process monitoring.

CHAPTER - 2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The project demonstrates the following operating system concepts: signal delivery and signal masking, process creation and process state management, signal atomicity and critical sections, async-signal-safe functions, race condition prevention in signal handlers, and the /proc filesystem. The Linux APIs used for these concepts are summarised below.

OS Concept / Linux API / System Call	Purpose in the Project
signal() / sigaction()	Register custom signal handlers for SIGUSR1, SIGUSR2, SIGALRM
sigprocmask() / sigblock()	Mask/unmask signals to prevent race conditions in critical sections
kill() / raise()	Send signals to processes (master to slave and vice versa)
fork() / exec()	Create and spawn slave worker processes
waitpid() / wait()	Monitor child process status and handle process termination
pause() / sleep()	Suspend process execution until signal delivery
/proc filesystem	Read process statistics (/proc/PID/stat, /proc/PID/status)
volatile sig_atomic_t	Use atomic types for signal-safe global variables

2.2 Existing Methods / Algorithms

Traditional IPC mechanisms such as pipes, message queues and sockets are reliable, but they introduce overhead when all that is needed is a simple event notification, a status update or a control message between processes. POSIX signals are a lightweight alternative, yet they are often underutilized or misunderstood, and developers struggle with writing reliable handlers, managing signal masks and delivery, coordinating processes with different priorities, and monitoring process states in real time.

2.3 Proposed Approach

The system architecture consists of three main components: (1) Master Process Controller, (2) Slave Worker Processes, and (3) Monitoring Dashboard. The master process spawns multiple slave processes and maintains signal handlers for SIGUSR1 (status request), SIGUSR2 (shutdown) and SIGALRM (periodic monitoring). Each slave process registers its own signal handlers and responds to control signals by updating its internal state.

Communication flow: the master sends SIGUSR1, the slave receives and processes it, the slave updates its status, and the master collects the results. Signal masking prevents race conditions during critical sections. The monitoring dashboard reads /proc/PID/stat for resource usage and

maintains statistics on signal delivery latency.

The implementation uses standard C with POSIX APIs. Major functions include signal_setup() for handler registration, process_signal_handler() for async-signal-safe processing, and status_collection() for metrics gathering. Edge cases like orphaned processes, signal coalescing and graceful shutdown are handled through coordinated SIGTERM delivery.

2.4 Advantages of the Proposed Approach

- Lightweight, event-driven communication with minimal overhead compared with pipes and message queues.
- Sub-millisecond signal delivery latency.
- Scalable coordination of up to 100 concurrent processes.
- Race conditions are avoided through signal masking and async-signal-safe handlers.
- Real-time visibility into CPU usage, memory consumption and signal statistics through the monitoring dashboard.
- Works on any Linux/POSIX platform because signals are supported in all Unix-like systems.

CHAPTER - 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Requirement	Specification
Processor	CPU	Any x86/x64, 1 GHz+
RAM	Memory	Min 512 MB, Recommended 2 GB+

3.2 Software Requirements

Component	Requirement	Specification
Operating System	Linux Kernel	Linux 3.0+ (Ubuntu/CentOS)
Compiler	GCC	GCC 4.8.5 or higher
Libraries	C Library	glibc 2.17+
Tools	Dev Tools	make, gdb, valgrind
IDE/Editor	Dev Environment	VS Code/Vim/Emacs

3.3 Operating System Environment – Ubuntu / WSL

The system is developed and tested on Linux / Ubuntu 20.04 LTS. Any Linux distribution with kernel 3.0 or higher (Ubuntu/CentOS) that provides POSIX signal support and the /proc filesystem can run the project.

3.4 Development Tools and Compilers

Tool / Platform / Software	Purpose
Linux / Ubuntu 20.04 LTS	Operating system for development and testing
C (C99/C11)	Primary programming language for implementation
GCC / Clang	C compiler with POSIX compliance
GDB / Valgrind	Debugging and memory profiling tools
GNU Make	Build automation and compilation
strace / ltrace	System call tracing and analysis

CHAPTER - 4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The system is written in standard C (C99/C11) using POSIX APIs. C gives direct access to Linux system calls, signal handling primitives and process-control functions, which makes it the natural choice for systems-level programming.

4.2 GCC Compiler

GCC (version 4.8.5 or higher) or Clang is used to compile the project with POSIX compliance. GNU Make automates the build process and compilation.

4.3 Linux System Calls

The project relies on the following Linux system calls and library functions:

- **signal() / sigaction():** register signal handlers.
- **sigprocmask():** block or unblock signals during critical sections.
- **kill() / raise():** send signals between processes.
- **fork() / exec():** create and spawn slave worker processes.
- **waitpid() / wait():** monitor child process status and handle termination.
- **pause() / sleep():** suspend execution until a signal arrives, or delay execution for process synchronization.

4.4 POSIX Threads / Synchronization

Synchronization in this project is achieved at the process level through signal masking with sigprocmask(), signal-safe global variables declared as volatile sig_atomic_t, and the pause() and sleep() calls, which coordinate parent and child processes.

4.5 Linux Commands and Utilities

- **make:** build automation.
- **gdb:** debugging of the master and slave processes.
- **valgrind:** memory profiling.
- **strace / ltrace:** system call tracing and analysis.
- **/proc filesystem:** /proc/PID/stat and /proc/PID/status are read to collect process statistics.

4.6 Other Tools / Libraries Used

- glibc 2.17 or higher (C library).
- VS Code / Vim / Emacs as the development editor.
- GitHub for hosting the source code with deployment and testing documentation.

CHAPTER - 5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The parent (master) process initializes the signal handlers (SIGUSR1, SIGUSR2), waits for user commands and sends signals to the child processes created with fork(). Each child process (Handler1, Handler2) sets up its own signal handlers, waits for signals, processes the signals it receives and updates the shared state. The processes communicate through kill(pid, SIGUSR1/SIGUSR2) and exchange state through shared data.

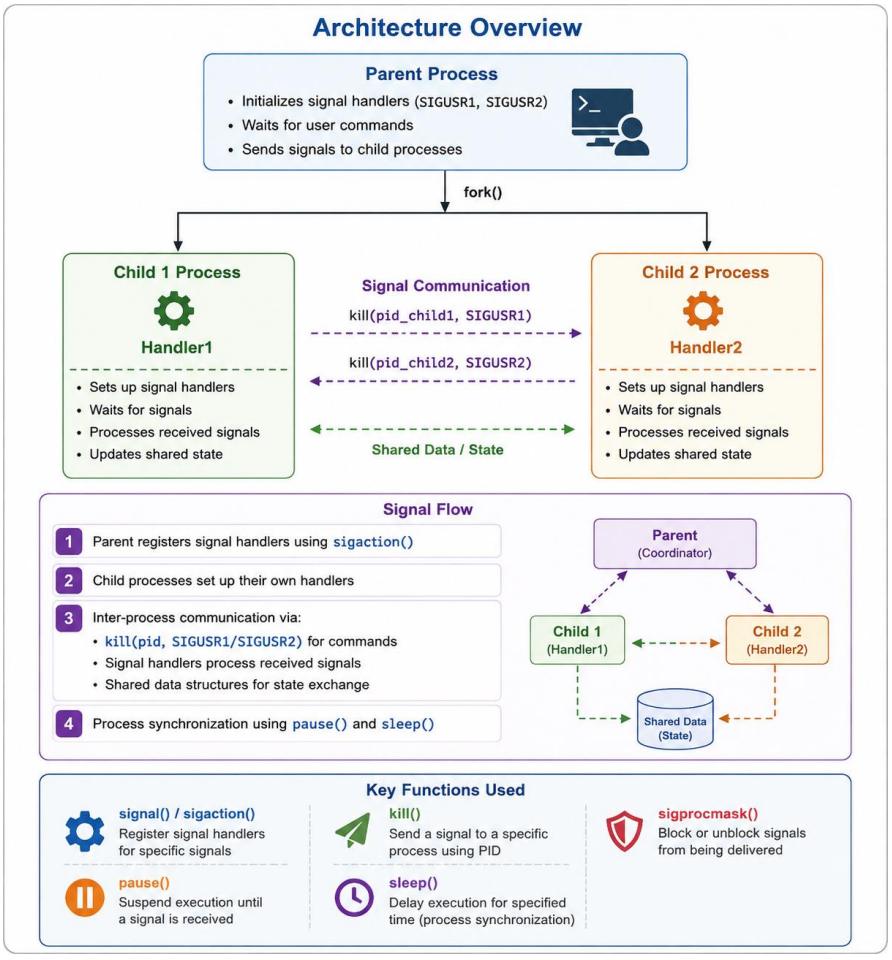


Figure 5.1: Architecture Overview of the Signal-Based Process Communication and Control System

5.2 System Workflow

1. The parent registers signal handlers using sigaction().
2. The child processes set up their own handlers.
3. Inter-process communication takes place via kill(pid, SIGUSR1/SIGUSR2) for commands, signal handlers that process the received signals, and shared data structures for state exchange.
4. Process synchronization is done using pause() and sleep().

In the monitoring flow, the master sends SIGUSR1, the slave receives and processes it, the slave

updates its status, and the master collects the results.

5.3 Flowchart / Data Flow

Data flows between the Parent (coordinator), Child 1 (Handler1) and Child 2 (Handler2) as shown in Figure 5.1. The parent communicates with both children through signals, the children signal each other with kill(), and both children read and update the shared data / state that holds process status information.

5.4 Module Description

- **Master Process Controller:** spawns the slave processes, maintains handlers for SIGUSR1 (status request), SIGUSR2 (shutdown) and SIGALRM (periodic monitoring), sends control signals and collects the results.
- **Slave Worker Processes:** register their signal handlers, respond to control signals by updating internal state, and report status to the master.
- **Monitoring Dashboard:** reads /proc/PID/stat for resource usage and maintains statistics on CPU usage, memory consumption and signal delivery latency.

Major functions: signal_setup() registers handlers, process_signal_handler() performs async-signal-safe processing, and status_collection() gathers metrics.

5.5 Input and Output Design

- **Input:** user commands to the parent process; control signals SIGUSR1, SIGUSR2, SIGALRM and SIGTERM; process statistics read from /proc/PID/stat and /proc/PID/status.
- **Output:** status reports from slave processes, and real-time dashboard output showing CPU usage, memory consumption, signal delivery statistics and latency.

CHAPTER - 6 IMPLEMENTATION

6.1 Module-wise Implementation

- **Master Process Controller** – spawns slaves, handles SIGUSR1/SIGUSR2/SIGALRM and coordinates shutdown.
- **Slave Worker Processes** – register handlers, update internal state and report status.
- **Monitoring Dashboard** – collects metrics through status_collection() and displays real-time statistics.

6.2 Algorithm / Pseudocode

To be added after implementation.

6.3 Core C / C++ Implementation

To be added after implementation.

6.4 Linux System Calls / APIs Used

signal() / sigaction(), sigprocmask(), kill() / raise(), fork() / exec(), waitpid() / wait(), pause() / sleep(), the /proc filesystem and volatile sig_atomic_t (see Table in Section 2.1).

6.5 Process / Thread Implementation

The master process creates the slave worker processes with fork() / exec() and monitors them with waitpid() / wait(). Orphaned processes are handled and slaves are shut down gracefully through coordinated SIGTERM delivery.

6.6 IPC / Synchronization Implementation

Communication between master and slaves uses kill() to deliver SIGUSR1, SIGUSR2 and SIGALRM. Signal masking with sigprocmask() protects critical sections and prevents race conditions, global state shared with handlers is declared volatile sig_atomic_t, and handlers use only async-signal-safe functions.

6.7 Memory Management Implementation

To be added after implementation.

6.8 Optional User Interface Implementation

To be added after implementation.

CHAPTER - 7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

To be added after implementation.

7.2 Execution Commands

To be added after implementation.

7.3 Sample Inputs and Outputs

To be added after implementation.

7.4 Screenshots of Execution

To be added after implementation.

7.5 Demonstration of OS Concepts

To be added after implementation.

CHAPTER - 8 CONCLUSION

8.1 Summary of the Project

The Linux Signal-Based IPC System demonstrates how POSIX signals can be used for efficient inter-process communication and process control. Signals such as SIGUSR1, SIGUSR2 and SIGALRM enable lightweight and event-driven communication between processes. The system uses signal handlers, signal masking and process synchronization to coordinate parent and child processes, and the master-slave architecture provides an effective approach for monitoring processes and managing their states.

Overall, signal-based IPC is a simple, lightweight and efficient solution for real-time process coordination in Linux/POSIX systems.

8.2 Objectives Achieved

- A signal handling framework for SIGUSR1, SIGUSR2, SIGALRM and SIGTERM with proper masking and delivery semantics.
- A master-slave architecture coordinated through signal-based communication.
- A process monitoring system based on the /proc filesystem and signal counters.
- Demonstration of signal atomicity, critical sections, async-signal-safe functions and race condition prevention.

8.3 Results Achieved

The expected results are: demonstration of correct signal handling semantics with sub-millisecond delivery latency; a working prototype showing master-slave coordination via signals, managing up to 100 concurrent slave processes; performance metrics showing the efficiency of signal-based IPC compared with traditional mechanisms; complete source code with inline documentation and example usage; and a technical report on signal handling best practices and lessons learned.

8.4 Skills and Learning Outcomes

This project provides practical understanding of Linux process management, signal delivery, IPC mechanisms and low-level system programming, along with experience in signal masking, async-signal-safe programming, race condition prevention, debugging with GDB, Valgrind and strace, and building efficient systems-level applications on Linux/POSIX platforms.

CHAPTER - 9 REFERENCES

9.1 Books

1. W. Richard Stevens and Stephen A. Rago, Advanced Programming in the UNIX Environment, 3rd Edition, Addison-Wesley, 2013.
2. Michael Kerrisk, The Linux Programming Interface, No Starch Press, 2010.

9.2 Research Papers

To be added.

9.3 Linux / GNU Documentation

1. Linux manual pages: signal(7), sigaction(2), sigprocmask(2), kill(2), fork(2), waitpid(2), pause(2), proc(5).
2. GNU C Library Reference Manual – Signal Handling.

9.4 Online Resources

1. The Linux man-pages project: https://man7.org/linux/man-pages/
2. GNU C Library manual: https://www.gnu.org/software/libc/manual/